boba_extension_dim03

Dimension 03: Browser Extension Architecture (Manifest V3)

Comprehensive Architecture Guide for Torrent Link Extraction Extension

Date: 2025-07-17 **Scope:** Manifest V3 browser extension for extracting torrent links from web pages and sending to external API **Browsers:** Chrome, Edge, Opera (Chromium-based), Firefox

---

Table of Contents

---

1. Manifest V3 Structure

1.1 Required Fields

The only mandatory keys in `manifest.json` are `"manifest_version"`, `"version"`, and `"name"` [77]. All other fields are optional but practically required for a functional extension.

```
{
  "manifest_version": 3,
  "name": "Torrent Link Extractor",
  "version": "1.0.0",
  "description":
        "Extracts torrent magnet links from web pages and sends
        them to your API server",
  "minimum_chrome_version": "120"
}
```

Claim: The `manifest_version`, `version`, and `name` are the only mandatory keys in manifest.json. **Source:** MDN Web Docs **URL:** https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json **Date:** 2026-05-06 **Excerpt:** `"manifest_version", "version", and "name" are the only mandatory keys.` **Confidence:** high

1.2 Complete Manifest for Torrent Extension

```
{
  "manifest_version": 3,
  "name": "Torrent Link Extractor",
  "version": "1.0.0",
  "description":
        "Extracts torrent magnet links from web pages and sends
        them to your API server",
  "minimum_chrome_version": "120",

  "permissions": [
    "storage",
    "notifications",
    "contextMenus",
    "scripting",
    "alarms",
    "activeTab"
  ],

  "optional_permissions": [
    "clipboardWrite"
  ],

  "host_permissions": [
```

```
    "http://*/",
    "https://*/"
  ],

  "optional_host_permissions": [
    "<all_urls>"
  ],

  "background": {
    "service_worker": "service-worker.js",
    "type": "module"
  },

  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    },
    "default_title": "Torrent Link Extractor",
    "default_popup": "popup.html"
  },

  "content_scripts": [
    {
      "matches": ["http://*/*", "https://*/*"],
      "js": ["content-scripts/torrent-scanner.js"],
      "run_at": "document_idle",
      "all_frames": true
    }
  ],

  "web_accessible_resources": [
    {
      "resources": ["assets/*", "injected-scripts/*"],
      "matches": ["http://*/*", "https://*/*"]
    }
  ],

  "icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  },

  "options_page": "options.html",

  "commands": {
    "_execute_action": {
      "suggested_key": {
```

```
        "default": "Alt+Shift+T"
      }
    },
    "extract-torrents": {
      "suggested_key": {
        "default": "Ctrl+Shift+E"
      },
      "description": "Extract torrent links from current page"
    }
  }
}
```

1.3 Key Manifest Fields Reference

Field	Required	Description
`manifest_version`	Yes	Set to `3` for MV3
`name`	Yes	Extension name (max 45 chars)
`version`	Yes	SemVer format: `major.minor.patch.build` [16]
`description`	Recommended	Brief description of functionality
`permissions`	No	API permissions (storage, notifications, etc.)
`host_permissions`	No	URL patterns for cross-origin access [63]
`optional_permissions`	No	Runtime-requested permissions
`background.service_worker`	No	Path to service worker JS file
`action`	No	Toolbar button config (replaces `browser_action`)
`content_scripts`	No	Static content script declarations
`web_accessible_resources`	No	Exposed extension files to web pages
`options_page`	No	Extension settings page
`commands`	No	Keyboard shortcuts

Claim: Manifest V3 uses `host_permissions` (separate from `permissions`) for URL pattern access, replacing the V2 approach of including URLs in the `permissions` array. **Source:** Chrome Extensions Documentation **URL:** https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions **Date:** 2024-02-05 **Excerpt:**
`"permissions": Contains items from a list of known strings...`
`"host_permissions": Contains one or more match patterns that give access to one or more hosts.` **Confidence:** high

---

2. Service Worker Lifecycle

2.1 Lifecycle Events

The extension service worker lifecycle follows this sequence [40] [71]:

```
INSTALL:    ServiceWorkerRegistration.install  (install event)
            chrome.runtime.onInstalled         (extension
installed/updated)
            ServiceWorkerRegistration.active   (activate event)

RUNTIME:    chrome.runtime.onStartup           (browser profile
startup)
            [Event-driven execution]

SHUTDOWN:   Idle 30 seconds → terminate
            Single request > 5 minutes → terminate
            fetch() response > 30 seconds → terminate
```

Claim: Chrome terminates the service worker after 30 seconds of inactivity, or when a single request exceeds 5 minutes, or when a fetch response takes longer than 30 seconds. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle **Date:** 2023-05-02 **Excerpt:** “Usually, Chrome will terminate a service worker when one of the following conditions is met: No operation for 30 seconds… When a single request takes more than 5 minutes… When a fetch() response takes more than 30 seconds to arrive.” **Confidence:** high

2.2 Version-Specific Behavior

Chrome Version	Behavior Change
Chrome 120	Alarms can be set to minimum 30-second period [40]

Chrome Version	Behavior Change
Chrome 118	debugger sessions keep service worker alive
Chrome 116	WebSocket connections extend lifecycle; some APIs can exceed 5-min timeout
Chrome 114	Persistent messaging (sendMessage) keeps SW alive; opening ports no longer resets timer
Chrome 110	Extension API calls reset the idle timer
Chrome 109	Messages from offscreen documents reset timer [40]

2.3 Keep-Alive Strategies

Strategy 1: Alarms API (Recommended)

```
// service-worker.js
import { checkAlarmState } from './alarms.js';

// On startup, ensure alarm exists
checkAlarmState();

async function checkAlarmState() {
  const alarm = await chrome.alarms.get('keep-alive');
  if (!alarm) {
    await chrome.alarms.create('keep-alive', {
      periodInMinutes: 0.5,  // 30 seconds (Chrome 120+)
      delayInMinutes: 0.1
    });
  }
}

chrome.alarms.onAlarm.addListener((alarm) => {
  if (alarm.name === 'keep-alive') {
    // Perform periodic tasks: check storage, update badge, etc.
    console.log('[TorrentExt] Keepalive heartbeat');
  }
});
```

Claim: The chrome.alarms API is the official recommended way to keep service workers alive for periodic tasks in MV3. Alarms wake up the service worker even after termination. **Source:** Chrome for Developers / Medium technical article **URL:** https://developer.chrome.com/docs/extensions/reference/api/alarms

Date: 2026-05-15 **Excerpt:** “Alarms wake up the service worker even after termination. They’re the official ‘please don’t forget about me’ API.” **Confidence:** high

Strategy 2: Long-Lived Port Connections

```
// service-worker.js
const keepalivePorts = new Set();

chrome.runtime.onConnect.addListener((port) => {
  if (port.name === 'torrent-ext-keepalive') {
    keepalivePorts.add(port);
    port.onDisconnect.addListener(() => {
      keepalivePorts.delete(port);
    });
  }
});
```

Strategy 3: waitUntil() Helper for Long Operations

```
// service-worker.js
async function waitUntil(promise) {
  const keepAlive = setInterval(chrome.runtime.getPlatformInfo,
        25 * 1000);
  try {
    await promise;
  } finally {
    clearInterval(keepAlive);
  }
}

// Usage for long-running API calls
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  if (request.type === 'EXTRACT_AND_SEND') {
    waitUntil(handleExtraction(request.payload))
      .then(result => sendResponse({ success: true, result }))
      .catch(error => sendResponse({ success: false, error:
        error.message }));
    return true; // Keep channel open
  }
});
```

Claim: For operations that may exceed 30 seconds (e.g., large file processing, slow API calls), a waitUntil() helper that periodically calls an extension API can keep the service worker alive. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/develop/migrate/to-service-workers **Date:** 2023-03-09 **Excerpt:** “The following example shows a waitUntil() helper that keeps the service worker alive until a given promise resolves…” **Confidence:** high

2.4 Critical Rule: Top-Level Listeners

```
//    CORRECT: Listeners registered at top level
chrome.runtime.onMessage.addListener(handleMessage);
chrome.runtime.onInstalled.addListener(handleInstalled);
chrome.alarms.onAlarm.addListener(handleAlarm);
chrome.notifications.onClicked.addListener(handleNotificationClick);

//    WRONG: Listeners inside async callbacks
async function init() {
  const config = await chrome.storage.local.get('config');
  // DON'T register listeners here - they won't survive restart!
  chrome.runtime.onMessage.addListener(handleMessage);
}
```

Claim: Event listeners must be registered synchronously at the top level of the service worker. Listeners registered inside async callbacks will not survive service worker restarts. **Source:** Extension.js documentation **URL:** https://extension.js.org/docs/concepts/manifest-v3 **Date:** 2026-04-22 **Excerpt:** “Register event listeners synchronously at the top of the file, not inside async callbacks. Listeners registered inside async callbacks will not fire on the wake-up event that loaded the worker.” **Confidence:** high

---

3. Content Scripts

3.1 Injection Timing (run_at)

Three timing options corresponding to `Document.readyState` [37] [38]:

`run_at` Value	readyState	When Script Runs
`document_start`	`loading`	DOM still loading; after CSS but before any DOM construction or other scripts
`document_end`	`interactive`	DOM complete, but subresources (images, frames) still loading
`document_idle` (default)	`complete`	Document and all resources finished loading

Claim: The `run_at` field controls when JavaScript files are injected and corresponds directly to `Document.readyState` values. **Source:** Chrome Extensions Documentation **URL:** https://developer.chrome.com/docs/extensions/reference/manifest/

content-scripts **Date:** 2023-08-10 **Excerpt:** `"document_start": DOM is still loading. "document_end": Subresources like images and frames are still loading. "document_idle": DOM and resources are complete.` **Confidence:** high

3.2 Recommended Strategy for Torrent Detection

For torrent link extraction, `document_idle` is optimal because: - Magnet links may be loaded dynamically via JavaScript after page load - Torrent tables and link lists often populate after AJAX calls - The DOM is fully parsed and stable

```
{
  "content_scripts": [
    {
      "matches": ["http://*/*", "https://*/*"],
      "js": ["content-scripts/torrent-scanner.js"],
      "run_at": "document_idle",
      "all_frames": true,
      "match_origin_as_fallback": false
    }
  ]
}
```

3.3 Match Patterns

Match patterns follow the syntax: `<scheme>://<host><path>` [100] [115]

Pattern	Matches
`http://*/*`	Any host, any path over HTTP
`https://*/*`	Any host, any path over HTTPS
`<all_urls>`	All supported schemes (http, https, ws, wss, ftp, data, file)
`*://*.example.com/*`	example.com and all subdomains
`*://*/`	All HTTP/HTTPS URLs (root path only)

Claim: The special `<all_urls>` pattern matches all URLs under any supported scheme. Individual scheme+host+path combinations provide more granular control. **Source:** Opera Help / MDN **URL:** https://help.opera.com/en/extensions/match-patterns/ **Date:** 2024-06-12 **Excerpt:** “The special pattern `<all_urls>` matches any URL that starts with a permitted scheme.” **Confidence:** high

3.4 Execution Worlds

Content scripts run in an isolated world by default. Two options exist [63] [74]:

World	Access	Use Case
`ISOLATED` (default)	Separate JS scope from page; shares DOM; has `chrome.runtime` API	Standard content script operation
`MAIN`	Shared JS scope with page; NO `chrome.runtime` API	Accessing page JS variables/functions

```
// Inject into MAIN world to access page's JavaScript variables
chrome.scripting.executeScript({
  target: { tabId: tab.id },
  func: extractTorrentsFromPage,
  world: 'MAIN'
});
```

Warning: Due to lack of isolation in `MAIN` world, the web page can detect and interfere with executed code [74].

3.5 Content Script for Torrent Detection

```
// content-scripts/torrent-scanner.js

// Magnet link regex patterns
const MAGNET_REGEX = /magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32,40}/
        gi;
const TORRENT_FILE_REGEX = /\.torrent($|\?|&)/i;
const TRACKER_REGEX = /(udp|http|https):\/\/[^\s"'<>]+\/(announce|
        scrape)/gi;

// State
let observer = null;
let foundMagnets = new Set();

// Scan the page for torrent links
function scanForTorrents() {
  const results = {
    magnets: [],
    torrentFiles: [],
    trackers: [],
    timestamp: Date.now(),
    url: window.location.href,
    title: document.title
  };
```

```
  // 1. Scan anchor hrefs
  const links = document.querySelectorAll('a[href]');
  links.forEach(link => {
    const href = link.href;

    if (href.startsWith('magnet:')) {
      const match = href.match(MAGNET_REGEX);
      if (match && !foundMagnets.has(match[0])) {
        foundMagnets.add(match[0]);
        results.magnets.push({
          link: match[0],
          text: link.textContent.trim(),
          title: link.title || ''
        });
      }
    }

    if (TORRENT_FILE_REGEX.test(href)) {
      results.torrentFiles.push({
        link: href,
        text: link.textContent.trim()
      });
    }
  });

  // 2. Scan text content for trackers
  const bodyText = document.body.innerText;
  const trackerMatches = bodyText.match(TRACKER_REGEX);
  if (trackerMatches) {
    results.trackers = [...new Set(trackerMatches)];
  }

  return results;
}

// Listen for messages from popup/service worker
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  if (request.action === 'SCAN_TORRENTS') {
    const results = scanForTorrents();
    sendResponse(results);
    return false;
  }

  if (request.action === 'GET_FOUND_TORRENTS') {
    sendResponse({
      magnets: [...foundMagnets],
      url: window.location.href
    });
    return false;
  }
});
```

```
// Observe DOM changes for dynamically loaded content
function startObserver() {
  observer = new MutationObserver((mutations) => {
    let hasNewLinks = false;
    mutations.forEach(mutation => {
      mutation.addedNodes.forEach(node => {
        if (node.nodeType === Node.ELEMENT_NODE) {
          if (node.matches?.('a[href^="magnet:"]') ||
              node.querySelector?.('a[href^="magnet:"]')) {
            hasNewLinks = true;
          }
        }
      });
    });

    if (hasNewLinks) {
      const results = scanForTorrents();
      if (results.magnets.length > 0) {
        // Notify background of new magnets
        chrome.runtime.sendMessage({
          type: 'NEW_MAGNETS_FOUND',
          data: results
        }).catch(() => {});
      }
    }
  });

  observer.observe(document.body, {
    childList: true,
    subtree: true
  });
}

// Initialize
document.addEventListener('DOMContentLoaded', () => {
  const results = scanForTorrents();
  if (results.magnets.length > 0) {
    chrome.runtime.sendMessage({
      type: 'TORRENTS_FOUND_ON_LOAD',
      data: results
    }).catch(() => {});
  }
  startObserver();
});

// If already loaded, scan immediately
if (document.readyState === 'complete') {
  const results = scanForTorrents();
  if (results.magnets.length > 0) {
    chrome.runtime.sendMessage({
      type: 'TORRENTS_FOUND_ON_LOAD',
```

```
      data: results
    }).catch(() => {});
  }
  startObserver();
}
```

---

4. Background Service Worker — Fetch API

4.1 Making External API Calls

The service worker uses the standard `fetch()` API to communicate with external servers:

```
// service-worker.js - API client

const API_CONFIG = {
  baseUrl: '',  // Loaded from storage
  apiKey: '',   // Loaded from storage
  timeout: 30000
};

// Load config from storage on startup
async function loadConfig() {
  const config = await chrome.storage.local.get(['apiBaseUrl',
        'apiKey', 'timeout']);
  API_CONFIG.baseUrl = config.apiBaseUrl || 'http://
        localhost:8080';
  API_CONFIG.apiKey = config.apiKey || '';
  API_CONFIG.timeout = config.timeout || 30000;
}
loadConfig();

/**
 * Send extracted torrent data to external API
 * @param {Object} payload - Torrent data from content script
 * @returns {Promise<Object>} API response
 */
async function sendToApi(endpoint, payload) {
  const url = `${API_CONFIG.baseUrl}${endpoint}`;
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(),
        API_CONFIG.timeout);

  try {
    const response = await fetch(url, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
```

```
        'X-API-Key': API_CONFIG.apiKey,
        'X-Extension-Version':
        chrome.runtime.getManifest().version
      },
      body: JSON.stringify(payload),
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: $
        {response.statusText}`);
    }

    return await response.json();
  } catch (error) {
    if (error.name === 'AbortError') {
      throw new Error('Request timed out');
    }
    throw error;
  }
}

// Handle extraction + send flow
async function handleExtraction(data) {
  // Keep service worker alive during operation
  const keepAlive = setInterval(() => {
    chrome.runtime.getPlatformInfo(() => {});
  }, 20000);

  try {
    // Send to API
    const result = await sendToApi('/api/torrents/extract', {
      source_url: data.url,
      title: data.title,
      magnets: data.magnets,
      torrent_files: data.torrentFiles,
      trackers: data.trackers,
      extracted_at: new Date().toISOString()
    });

    // Show success notification
    chrome.notifications.create({
      type: 'basic',
      iconUrl: 'icons/icon-48.png',
      title: 'Torrents Extracted',
      message: `${data.magnets.length} magnet(s) sent to API
        successfully`
    });

    return result;
```

```
  } catch (error) {
    // Show error notification
    chrome.notifications.create({
      type: 'basic',
      iconUrl: 'icons/icon-48.png',
      title: 'Extraction Failed',
      message: error.message
    });
    throw error;
  } finally {
    clearInterval(keepAlive);
  }
}
```

4.2 Fetch Timeout Handling

Claim: The fetch() API has a default timeout of 30 seconds for responses in Chrome extension service workers, after which the service worker may be terminated. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle **Date:** 2023-05-02 **Excerpt:** “When a fetch() response takes more than 30 seconds to arrive.” **Confidence:** high

Use `AbortController` for explicit timeout control:

```
async function fetchWithTimeout(url, options = {}, timeoutMs =
        30000) {
  const controller = new AbortController();
  const id = setTimeout(() => controller.abort(), timeoutMs);

  try {
    const response = await fetch(url, {
      ...options,
      signal: controller.signal
    });
    clearTimeout(id);
    return response;
  } catch (error) {
    clearTimeout(id);
    if (error.name === 'AbortError') {
      throw new Error(`Request timeout after ${timeoutMs}ms`);
    }
    throw error;
  }
}
```

---

5. Messaging APIs

5.1 One-Time Messages

Content Script → Background

```
// content-script.js (sending)
const response = await chrome.runtime.sendMessage({
  type: 'SEND_TORRENTS',
  data: torrentData
});
console.log('Background response:', response);

// service-worker.js (receiving)
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  if (request.type === 'SEND_TORRENTS') {
    handleExtraction(request.data)
      .then(result => sendResponse({ success: true, result }))
      .catch(error => sendResponse({ success: false, error:
        error.message }));
    return true; // CRITICAL: Keep channel open for async response
  }
});
```

Claim: `return true` from the onMessage listener is required when sending an asynchronous response. Without it, the message channel closes before the async operation completes. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/develop/concepts/messaging **Date:** 2025-12-03 **Excerpt:** “Both of these APIs return a Promise that resolves to the response from the recipient.” **Confidence:** high

Popup → Content Script (via tabs.sendMessage)

```
// popup.js - Send to specific tab's content script
async function scanCurrentPage() {
  const [tab] = await chrome.tabs.query({ active: true,
        currentWindow: true });

  const results = await chrome.tabs.sendMessage(tab.id, {
    action: 'SCAN_TORRENTS'
  });

  displayResults(results);
}
```

Popup → Background

```
// popup.js
chrome.runtime.sendMessage({ type: 'GET_CONFIG' })
  .then(response => console.log('Config:', response));
```

5.2 Long-Lived Connections (Ports)

For real-time communication (e.g., streaming extraction progress):

```
// popup.js - Create port connection
const port = chrome.runtime.connect({ name: 'torrent-
        extraction' });

port.postMessage({ action: 'START_EXTRACTION', tabId:
        currentTabId });

port.onMessage.addListener((msg) => {
  if (msg.type === 'PROGRESS') {
    updateProgressBar(msg.percent);
  }
  if (msg.type === 'COMPLETE') {
    showResults(msg.results);
  }
});

port.onDisconnect.addListener(() => {
  console.log('Port disconnected');
});

// service-worker.js - Handle port connections
chrome.runtime.onConnect.addListener((port) => {
  if (port.name === 'torrent-extraction') {
    port.onMessage.addListener(async (msg) => {
      if (msg.action === 'START_EXTRACTION') {
        try {
          const results = await extractAndSend(msg.tabId,
        (progress) => {
            port.postMessage({ type: 'PROGRESS', percent:
        progress });
          });
          port.postMessage({ type: 'COMPLETE', results });
        } catch (error) {
          port.postMessage({ type: 'ERROR', message:
        error.message });
        }
      }
    });
  }
});
```

Claim: Chrome.runtime.connect() creates a reusable long-lived message passing channel with a runtime.Port object on each end. Port connections from UI/content scripts signal Chrome that the worker is still active. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/develop/concepts/messaging **Date:** 2025-12-03 **Excerpt:** "To create a reusable long-lived message passing channel, call runtime.connect()... When establishing a connection, each end is assigned a runtime.Port object..." **Confidence:** high

5.3 Cross-Extension Messaging

```
// Allow other extensions to communicate
chrome.runtime.onMessageExternal.addListener((request, sender,
        sendResponse) => {
  if (request.action === 'EXTRACT_TORRENTS') {
    // Validate sender if needed
    scanAllTabs().then(sendResponse);
    return true;
  }
});
```

5.4 Message Type Constants

```
// shared/messages.js - Centralized message types
export const MSG_TYPES = {
  // Content → Background
  TORRENTS_FOUND: 'TORRENTS_FOUND',
  NEW_MAGNETS_FOUND: 'NEW_MAGNETS_FOUND',

  // Popup → Background
  GET_CONFIG: 'GET_CONFIG',
  SET_CONFIG: 'SET_CONFIG',
  SEND_TORRENTS: 'SEND_TORRENTS',
  GET_STATS: 'GET_STATS',

  // Background → Content
  SCAN_TORRENTS: 'SCAN_TORRENTS',
  GET_FOUND_TORRENTS: 'GET_FOUND_TORRENTS',

  // Internal
  PROGRESS: 'PROGRESS',
  COMPLETE: 'COMPLETE',
  ERROR: 'ERROR'
};
```

---

6. chrome.storage API

6.1 Storage Areas

Four storage areas available in MV3 [35]:

Area	Scope	Quota	Persistence	Use Case
`storage.local`	Local to device	~10 MB (unlimitedStorage to increase)	Cleared on extension removal	Large data, user settings
`storage.sync`	Synced across Chrome profiles	~100 KB total, 8 KB per item	Synced via Google account	Cross-device user settings
`storage.session`	In-memory, session only	~10 MB	Lost on browser close	Service worker runtime state
`storage.managed`	Admin-configured	N/A	Read-only	Enterprise policies

Claim: storage.local has ~10MB quota, storage.sync has ~100KB, storage.session has ~10MB. storage.session data is lost when the browser session ends. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/mv2/reference/storage **Date:** 2025-12-19 **Excerpt:** “storage.local: Quota limit of about 10 MB… storage.sync: Quota limit of about 100 KB, 8 KB per item… storage.session: Quota limit of about 10 MB…” **Confidence:** high

6.2 Storage Schema for Torrent Extension

```
// Storage schema design
const DEFAULT_CONFIG = {
  // Server configuration
  apiBaseUrl: 'http://localhost:8080',
  apiKey: '',
  timeout: 30000,

  // Feature toggles
  autoExtract: false,
  showNotifications: true,
  notifyOnNewMagnets: true,

  // UI preferences
  badgeDisplay: 'count',  // 'count' | 'none'

  // Statistics
```

```
  totalExtracted: 0,
  totalSent: 0,
  lastExtraction: null
};

// Initialize storage on install
chrome.runtime.onInstalled.addListener((details) => {
  if (details.reason === 'install') {
    chrome.storage.local.set({ config: DEFAULT_CONFIG });
  }
});
```

6.3 Storage Operations

```
// Get config
async function getConfig() {
  const { config } = await chrome.storage.local.get('config');
  return { ...DEFAULT_CONFIG, ...config };
}

// Update config
async function updateConfig(updates) {
  const { config } = await chrome.storage.local.get('config');
  await chrome.storage.local.set({
    config: { ...config, ...updates }
  });
}

// Listen for storage changes (sync across contexts)
chrome.storage.onChanged.addListener((changes, areaName) => {
  if (areaName === 'local' && changes.config) {
    // Config changed - update in-memory cache
    console.log('Config updated:', changes.config.newValue);
  }
});
```

6.4 Using storage.session for Service Worker State

```
// Use session storage for transient state that shouldn't persist
async function cacheTorrentData(tabId, data) {
  await chrome.storage.session.set({
    [`torrents_${tabId}`]: data
  });
}

async function getCachedTorrentData(tabId) {
  const result = await chrome.storage.session.get(`torrents_$
        {tabId}`);
  return result[`torrents_${tabId}`];
}
```

6.5 Quota Checking

```
async function checkStorageQuota() {
  const localBytes = await chrome.storage.local.getBytesInUse();
  const sessionBytes = await
        chrome.storage.session.getBytesInUse();

  console.log(`Local: ${(localBytes / 1024).toFixed(2)} KB / ~10
        MB`);
  console.log(`Session: ${(sessionBytes / 1024).toFixed(2)} KB /
        ~10 MB`);

  return { localBytes, sessionBytes };
}
```

---

7. chrome.action API

7.1 Manifest Declaration

```
{
  "action": {
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png",
      "48": "icons/icon-48.png",
      "128": "icons/icon-128.png"
    },
    "default_title": "Torrent Link Extractor - Click to view",
    "default_popup": "popup.html"
  }
}
```

7.2 Badge API (Show torrent count on icon)

```
// service-worker.js

// Update badge with torrent count
async function updateBadge(tabId, count) {
  if (count > 0) {
    await chrome.action.setBadgeText({
      text: count > 99 ? '99+' : String(count),
      tabId
    });
    await chrome.action.setBadgeBackgroundColor({
      color: '#4CAF50',  // Green for found torrents
      tabId
    });
  } else {
    await chrome.action.setBadgeText({ text: '', tabId });
```

```
  }
}

// Clear badge
async function clearBadge(tabId) {
  await chrome.action.setBadgeText({ text: '', tabId });
}

// Handle badge on tab navigation
chrome.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
  if (changeInfo.status === 'loading') {
    clearBadge(tabId);
  }
});
```

Claim: The chrome.action API provides setBadgeText() and setBadgeBackgroundColor() for displaying status information on the toolbar icon. Badge text should not exceed 4 characters. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/reference/api/action **Date:** 2025-08-11 **Excerpt:** "A badge is a small piece of text that is overlayed on the icon... We recommend badge text contains no more than 4 characters." **Confidence:** high

7.3 Click Handler (when no popup is set)

```
// If default_popup is NOT set in manifest
chrome.action.onClicked.addListener(async (tab) => {
  // Requires activeTab permission
  const results = await chrome.tabs.sendMessage(tab.id, {
    action: 'SCAN_TORRENTS'
  });

  await sendToApi(results);
  updateBadge(tab.id, results.magnets.length);
});
```

7.4 Dynamic Popup Setting

```
// Disable popup on certain pages
chrome.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
  if (tab.url?.startsWith('chrome://')) {
    chrome.action.setPopup({ popup: '', tabId }); // No popup
  } else {
    chrome.action.setPopup({ popup: 'popup.html', tabId });
  }
});
```

7.5 Context Menus

```
// Create context menus on install
chrome.runtime.onInstalled.addListener(() => {
  chrome.contextMenus.create({
    id: 'extract-torrents',
    title: 'Extract torrent links from this page',
    contexts: ['page', 'action']
  });

  chrome.contextMenus.create({
    id: 'send-to-api',
    title: 'Send found torrents to API',
    contexts: ['action']
  });

  chrome.contextMenus.create({
    id: 'separator-1',
    type: 'separator',
    contexts: ['action']
  });

  chrome.contextMenus.create({
    id: 'open-options',
    title: 'Options',
    contexts: ['action']
  });
});

// Handle context menu clicks
chrome.contextMenus.onClicked.addListener((info, tab) => {
  switch (info.menuItemId) {
    case 'extract-torrents':
      chrome.tabs.sendMessage(tab.id, { action:
        'SCAN_TORRENTS' });
      break;
    case 'send-to-api':
      sendCachedTorrentsToApi(tab.id);
      break;
    case 'open-options':
      chrome.runtime.openOptionsPage();
      break;
  }
});
```

Claim: Context menus should be created inside a `runtime.onInstalled` listener, and `contextMenus.onClicked` must be used to handle clicks from event pages instead of the `onclick` parameter. **Source:** Firefox Extension Workshop **URL:** https://extensionworkshop.com/documentation/develop/manifest-v3-migration-guide/ **Date:** 2026-03-22 **Excerpt:** “Place menu creation using menus.create or its alias contextMenus.create in a

runtime.onInstalled listener… the menus.onClicked event or its alias contextMenus.onClicked must be used to handle menu entry click events from an event page.” **Confidence:** high

---

8. chrome.notifications API

8.1 Permissions

```
{
  "permissions": ["notifications"]
}
```

No user-facing warning is shown for this permission [86].

8.2 Notification Templates

Four template types available [88]:

```
// 1. Basic notification
chrome.notifications.create('torrent-found', {
  type: 'basic',
  iconUrl: 'icons/icon-128.png',
  title: 'Torrent Links Found',
  message: `Found ${count} magnet link(s) on this page`,
  priority: 1
});

// 2. List notification
chrome.notifications.create('torrent-list', {
  type: 'list',
  iconUrl: 'icons/icon-128.png',
  title: 'Extracted Torrents',
  message: `${torrents.length} items found`,
  items: torrents.slice(0, 5).map(t => ({
    title: t.name || 'Unknown',
    message: t.link.substring(0, 60) + '...'
  }))
});

// 3. Progress notification
chrome.notifications.create('upload-progress', {
  type: 'progress',
  iconUrl: 'icons/icon-128.png',
  title: 'Sending to API...',
  message: 'Uploading torrent data',
  progress: 45
});
```

```
// 4. Notification with action buttons
chrome.notifications.create('send-complete', {
  type: 'basic',
  iconUrl: 'icons/icon-128.png',
  title: 'Upload Complete',
  message: `${count} torrents sent to API`,
  buttons: [
    { title: 'View Results' },
    { title: 'Dismiss' }
  ],
  requireInteraction: false
});
```

Claim: MV3 no longer accepts Base64 data URLs for notification icons. The icon must be a file path relative to the extension root. **Source:** Stack Overflow / Chrome Extension Guide **URL:** https://stackoverflow.com/questions/79808540/how-to-display-notifications-in-manifest-v3-service-worker **Date:** 2025-11-16 **Excerpt:** “Manifest V3 no longer accepts Base64 data URLs for the icon. You have to download the image you want and put it in your extension folder…” **Confidence:** high

8.3 Notification Event Handlers

```
// Handle notification clicks
chrome.notifications.onClicked.addListener((notificationId) => {
  if (notificationId === 'send-complete') {
    chrome.tabs.create({ url: `${API_CONFIG.baseUrl}/torrents` });
  }
  chrome.notifications.clear(notificationId);
});

// Handle button clicks
chrome.notifications.onButtonClicked.addListener((notificationId,
        buttonIndex) => {
  if (buttonIndex === 0) {
    // "View Results" clicked
    chrome.tabs.create({ url: `${API_CONFIG.baseUrl}/torrents` });
  }
  chrome.notifications.clear(notificationId);
});

// Handle closure
chrome.notifications.onClosed.addListener((notificationId,
        byUser) => {
  console.log(`Notification ${notificationId} closed by $
        {byUser ? 'user' : 'system'}`);
});
```

8.4 Notification Options Reference

Property	Type	Required	Description
`type`	string	Yes	`basic`, `image`, `list`, `progress`
`iconUrl`	string	Yes	Path to icon (relative to extension root)
`title`	string	Yes	Primary title
`message`	string	Yes	Body text
`contextMessage`	string	No	Smaller additional text
`priority`	number	No	`-2` to `2`, default `0`
`buttons`	array	No	Max 2 action buttons
`requireInteraction`	boolean	No	Stay visible until dismissed
`silent`	boolean	No	Suppress sound

---

9. chrome.permissions API

9.1 Permission Categories

MV3 separates permissions into distinct categories [63]:

Category	Key	When Granted
Required permissions	`permissions`	At install time
Host permissions	`host_permissions`	At install time (Chrome), may need manual enable (Firefox)
Optional permissions	`optional_permissions`	At runtime, via user gesture
Optional host permissions	`optional_host_permissions`	At runtime, via user gesture

9.2 Permission Model for Torrent Extension

```
{
  "permissions": [
    "storage",
    "notifications",
    "contextMenus",
    "scripting",
    "alarms",
```

```
    "activeTab"
  ],
  "host_permissions": [
    "http://*/",
    "https://*/"
  ],
  "optional_permissions": [
    "clipboardWrite"
  ]
}
```

9.3 activeTab Permission

The `activeTab` permission grants temporary access to the currently active tab only when invoked via a user gesture (clicking the action button, context menu, or keyboard shortcut) [63].

Advantages of activeTab: - No persistent host permission warnings at install - Only grants access when user explicitly triggers the extension - Automatically grants `scripting` permission on the active tab

9.4 Runtime Permission Requests

```
// Request additional permissions at runtime (requires user
        gesture)
async function requestBroadAccess() {
  const granted = await chrome.permissions.request({
    permissions: ['clipboardWrite'],
    origins: ['<all_urls>']
  });
  return granted;
}

// Check current permissions
async function hasPermission(origin) {
  return await chrome.permissions.contains({
    origins: [origin]
  });
}

// Remove permissions when no longer needed
async function removePermission(origin) {
  await chrome.permissions.remove({
    origins: [origin]
  });
}
```

Claim: `chrome.permissions.request()` must be called during a user gesture (e.g., inside a click handler). Calling it asynchronously (e.g., after message passing) will fail. **Source:** Extension Ninja

URL: https://www.extension.ninja/blog/post/solved-this-function-must-be-called-during-a-user-gesture/ **Date:** 2022-02-12 **Excerpt:** “User gesture is any user interaction that generates an event in JavaScript code… The key word here is ‘synchronously’. If code is executed asynchronously, even if it was called from the user gesture handler, it no longer executes in the same context.” **Confidence:** high

9.5 declarativeContent for Conditional Activation

```
// Use declarativeContent to show action only on pages with
        torrent links
chrome.runtime.onInstalled.addListener(() => {
  chrome.action.disable();

  chrome.declarativeContent.onPageChanged.removeRules(undefined, () =>
        {
    const rule = {
      conditions: [
        new chrome.declarativeContent.PageStateMatcher({
          css: ["a[href^='magnet:']"]
        })
      ],
      actions: [new chrome.declarativeContent.ShowAction()]
    };

    chrome.declarativeContent.onPageChanged.addRules([rule]);
  });
});
```

Claim: The declarativeContent API enables showing the extension action only when certain page conditions are met (e.g., CSS selectors match), without requiring persistent host permissions. **Source:** Chrome for Developers **URL:** https://developer.chrome.com/docs/extensions/reference/api/declarativeContent **Date:** 2025-08-11 **Excerpt:** “Rules consist of conditions and actions. If any of the conditions is met, all actions are taken… `PageStateMatcher` matches web pages if and only if all listed conditions are met.” **Confidence:** high

---

10. Offscreen Documents

10.1 Purpose

Offscreen documents provide a temporary DOM-capable page environment for MV3 extensions that need DOM APIs unavailable in service workers [52] [67].

Use cases for torrent extension: - Parsing HTML responses with `DOMParser` - Using `XMLHttpRequest` (though `fetch()` is available in SW) - Clipboard operations - Running third-party libraries that require DOM

Claim: Offscreen documents allow Manifest V3 extensions to access DOM-related features and APIs not available in service workers. Each extension can have one offscreen document per profile. **Source:** Chrome Developer Blog **URL:** https://developer.chrome.com/blog/Offscreen-Documents-in-Manifest-v3 **Date:** 2023-01-25 **Excerpt:** “Offscreen documents provide a temporary, headless page environment that allows an extension to leverage DOM capabilities in the background.” **Confidence:** high

10.2 Manifest Declaration

```
{
  "permissions": ["offscreen"]
}
```

10.3 Creating an Offscreen Document

```
// service-worker.js
const OFFSCREEN_DOCUMENT_PATH = '/offscreen/offscreen.html';
let creatingOffscreen;

async function setupOffscreenDocument(path) {
  // Check if document already exists
  if (await hasOffscreenDocument()) {
    return;
  }

  if (creatingOffscreen) {
    await creatingOffscreen;
  } else {
    creatingOffscreen = chrome.offscreen.createDocument({
      url: path,
      reasons: ['DOM_PARSER'],
      justification: 'Parse HTML responses to extract torrent
        metadata'
    });
    await creatingOffscreen;
    creatingOffscreen = null;
  }
}

async function hasOffscreenDocument() {
  const matchedClients = await clients.matchAll();
  return matchedClients.some(
    (c) => c.url ===
        chrome.runtime.getURL(OFFSCREEN_DOCUMENT_PATH)
```

```
  );
}

async function closeOffscreenDocument() {
  if (await hasOffscreenDocument()) {
    await chrome.offscreen.closeDocument();
  }
}
```

10.4 Offscreen Document HTML and Script

```
<!-- offscreen/offscreen.html -->
<!DOCTYPE html>
<html>
<head>
  <script src="offscreen.js" type="module"></script>
</head>
<body>
</body>
</html>

// offscreen/offscreen.js
chrome.runtime.onMessage.addListener((message, sender,
        sendResponse) => {
  if (message.target !== 'offscreen') return;

  if (message.type === 'PARSE_HTML') {
    parseHtmlForTorrents(message.html)
      .then(result => sendResponse(result))
      .catch(error => sendResponse({ error: error.message }));
    return true;
  }

  if (message.type === 'PARSE_TORRENT_FILE') {
    parseTorrentFile(message.buffer)
      .then(result => sendResponse(result))
      .catch(error => sendResponse({ error: error.message }));
    return true;
  }
});

async function parseHtmlForTorrents(html) {
  const parser = new DOMParser();
  const doc = parser.parseFromString(html, 'text/html');

  const magnets = [];
  doc.querySelectorAll('a[href^="magnet:"]').forEach(a => {
    magnets.push({
      link: a.href,
      text: a.textContent.trim()
    });
```

```
  });

  return { magnets };
}
```

10.5 Offscreen Document Reasons

Available `chrome.offscreen.Reason` values [67]:

Reason	Purpose
`DOM_PARSER`	Use DOMParser API
`DOM_SCRAPING`	Embed iframe and scrape DOM
`CLIPBOARD`	Interact with Clipboard API
`BLOBS`	Work with Blob objects
`LOCAL_STORAGE`	Access localStorage
`AUDIO_PLAYBACK`	Play audio
`IFRAME_SCRIPTING`	Script iframe contents
`WEB_RTC`	Use WebRTC APIs
`GEOLOCATION`	Use navigator.geolocation
`MATCH_MEDIA`	Use window.matchMedia
`BATTERY_STATUS`	Use navigator.getBattery
`WORKERS`	Spawn workers
`DISPLAY_MEDIA`	Use getDisplayMedia
`USER_MEDIA`	Use getUserMedia

---

11. Fetch API — CORS Considerations

11.1 CORS in Extension Context

Chrome extensions can bypass certain CORS restrictions through declared `host_permissions` [59]:

```
{
  "host_permissions": [
    "https://api.example.com/*"
  ]
}
```

Claim: By declaring host_permissions in the manifest, the extension's service worker can make cross-origin fetch requests that would normally be blocked by CORS. However, the target server must still send appropriate CORS headers. **Source:** Reintech Blog **URL:** https://reintech.io/blog/cors-chrome-

extensions **Date:** 2026-03-24 **Excerpt:** “Chrome extensions can request cross-origin resources through two mechanisms: Declared permissions in the manifest file… Background scripts that act as privileged intermediaries for content scripts.” **Confidence:** high

11.2 Content Script → Background Proxy Pattern

Content scripts cannot make cross-origin requests directly. Route through the service worker:

```
// content-script.js (cannot fetch directly)
const response = await chrome.runtime.sendMessage({
  type: 'API_REQUEST',
  payload: {
    url: 'https://api.example.com/torrents',
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: torrentData
  }
});

// service-worker.js (can fetch with host_permissions)
chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  if (request.type === 'API_REQUEST') {
    const { url, method, headers, body } = request.payload;

    fetch(url, {
      method: method || 'GET',
      headers: {
        'Content-Type': 'application/json',
        ...headers
      },
      body: body ? JSON.stringify(body) : null
    })
    .then(response => {
      if (!response.ok) throw new Error(`HTTP ${response.status}
        `);
      return response.json();
    })
    .then(data => sendResponse({ success: true, data }))
    .catch(error => sendResponse({ success: false, error:
        error.message }));

    return true; // Async response
  }
});
```

11.3 Origin Header Behavior

Important: Chrome adds an `Origin: chrome-extension://<extension-id>` header to fetch requests from the service worker. This cannot be overridden via the `headers` option [69].

```
// Server-side: Allow chrome-extension:// origins
// Express.js example:
app.use((req, res, next) => {
  const origin = req.headers.origin;
  if (origin?.startsWith('chrome-extension://')) {
    res.header('Access-Control-Allow-Origin', origin);
    res.header('Access-Control-Allow-Methods', 'GET, POST, PUT,
        DELETE');
    res.header('Access-Control-Allow-Headers', 'Content-Type,
        Authorization, X-API-Key');
  }
  next();
});
```

11.4 Fetch with Authentication

```
async function authenticatedFetch(url, options = {}) {
  const config = await getConfig();

  return fetch(url, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${config.apiKey}`,
      'X-Extension-Version': chrome.runtime.getManifest().version,
      ...options.headers
    }
  });
}
```

---

12. Extension Packaging

12.1 ZIP Structure

The extension package is a signed ZIP file with `.crx` extension [61]:

```
my-extension/
|-- manifest.json
|-- service-worker.js
|-- popup.html
|-- popup.js
```

```
|-- popup.css
|-- options.html
|-- options.js
|-- content-scripts/
|   |-- torrent-scanner.js
|-- offscreen/
|   |-- offscreen.html
|   |-- offscreen.js
|-- icons/
|   |-- icon-16.png
|   |-- icon-32.png
|   |-- icon-48.png
|   |-- icon-128.png
|-- assets/
|   |-- logo.png
|-- _locales/
    |-- en/
        |-- messages.json
```

12.2 CRX Package Format

CRX files are signed ZIP archives [61]:

1. First-time packaging generates a `.pem` private key file
2. The `.crx` file is the installable extension
3. The `.pem` file must be kept secure for future updates
4. Chrome generates the extension ID from a hash of the public key

12.3 Packaging via Chrome UI

```
1. Navigate to chrome://extensions
2. Enable Developer Mode
3. Click "Pack extension"
4. Select extension root directory
5. (For updates) Select existing .pem private key
6. Click "Pack Extension"
```

12.4 Update Mechanism (update.xml)

For self-hosted extensions, an XML update manifest is required [120]:

```
<?xml version='1.0' encoding='UTF-8'?>
<gupdate xmlns='http://www.google.com/update2/response'
         protocol='2.0'>
  <app appid='aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'>
    <updatecheck codebase='https://example.com/extension_v2.crx'
         version='2.0' />
```

```
  </app>
</gupdate>
```

Claim: The update manifest XML uses the Omaha format. The `appid` is the extension ID (hash of public key), `codebase` is the CRX URL, and `version` must match the manifest.json version. **Source:** Chromium Source Documentation **URL:** https://chromium.googlesource.com/chromium/src.git/+/d81f948395387efe97492bdec8efc65613cb3f88/chrome/common/extensions/docs/static/autoupdate.html **Date:** N/A **Excerpt:** “appid: The extension id, generated based on a hash of the extension’s public key… codebase: A URL to the crx file… version: This is used by the client to determine whether it should download the crx file.” **Confidence:** high

12.5 Manifest update_url

```
{
  "update_url": "https://example.com/updates.xml"
}
```

Chrome checks for updates every few hours. For testing, use `--extensions-update-frequency=45` flag.

---

13. Complete Reference Implementation

13.1 Service Worker (service-worker.js)

```
// service-worker.js - Main background script
import { MSG_TYPES } from './shared/messages.js';

// ==================== CONFIGURATION ====================

let API_CONFIG = {
  baseUrl: 'http://localhost:8080',
  apiKey: '',
  timeout: 30000
};

async function loadConfig() {
  const { config } = await chrome.storage.local.get('config');
  if (config) {
    API_CONFIG = { ...API_CONFIG, ...config };
  }
}

// ==================== LIFECYCLE ====================
```

```
chrome.runtime.onInstalled.addListener((details) => {
  if (details.reason === 'install') {
    chrome.storage.local.set({
      config: {
        apiBaseUrl: 'http://localhost:8080',
        apiKey: '',
        timeout: 30000,
        autoExtract: false,
        showNotifications: true,
        badgeDisplay: 'count',
        totalExtracted: 0,
        totalSent: 0
      }
    });
  }

  // Setup context menus
  setupContextMenus();

  // Setup alarm for periodic tasks
  setupKeepAliveAlarm();
});

chrome.runtime.onStartup.addListener(() => {
  loadConfig();
  setupKeepAliveAlarm();
});

// Load config immediately on SW start (top-level)
loadConfig();

// ==================== ALARMS ====================

async function setupKeepAliveAlarm() {
  const alarm = await chrome.alarms.get('keep-alive');
  if (!alarm) {
    await chrome.alarms.create('keep-alive', {
      periodInMinutes: 0.5,
      delayInMinutes: 0.1
    });
  }
}

chrome.alarms.onAlarm.addListener((alarm) => {
  if (alarm.name === 'keep-alive') {
    // Periodic cleanup or status check
    cleanupOldSessionData();
  }
});

// ==================== CONTEXT MENUS ====================
```

```
function setupContextMenus() {
  chrome.contextMenus.create({
    id: 'extract-torrents',
    title: 'Extract torrent links',
    contexts: ['page', 'action']
  });

  chrome.contextMenus.create({
    id: 'open-options',
    title: 'Options',
    contexts: ['action']
  });
}

chrome.contextMenus.onClicked.addListener((info, tab) => {
  if (info.menuItemId === 'extract-torrents') {
    chrome.tabs.sendMessage(tab.id, { action: 'SCAN_TORRENTS' });
  } else if (info.menuItemId === 'open-options') {
    chrome.runtime.openOptionsPage();
  }
});

// ==================== MESSAGING ====================

chrome.runtime.onMessage.addListener((request, sender,
        sendResponse) => {
  switch (request.type) {
    case MSG_TYPES.SEND_TORRENTS:
      handleExtraction(request.data)
        .then(result => sendResponse({ success: true, result }))
        .catch(error => sendResponse({ success: false, error:
        error.message }));
      return true; // Async

    case MSG_TYPES.GET_CONFIG:
      getConfig().then(sendResponse);
      return true;

    case MSG_TYPES.SET_CONFIG:
      updateConfig(request.data).then(() => sendResponse({
        success: true }));
      return true;

    case MSG_TYPES.GET_STATS:
      getStats().then(sendResponse);
      return true;

    case MSG_TYPES.TORRENTS_FOUND:
    case MSG_TYPES.TORRENTS_FOUND_ON_LOAD:
      handleFoundTorrents(request.data, sender.tab?.id);
      sendResponse({ received: true });
```

```
      return false;

    default:
      return false;
  }
});

// ==================== API COMMUNICATION ====================

async function sendToApi(endpoint, payload) {
  const url = `${API_CONFIG.baseUrl}${endpoint}`;
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(),
        API_CONFIG.timeout);

  try {
    const response = await fetch(url, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'X-API-Key': API_CONFIG.apiKey,
        'X-Extension-Version':
        chrome.runtime.getManifest().version
      },
      body: JSON.stringify(payload),
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: $
        {response.statusText}`);
    }

    return await response.json();
  } catch (error) {
    clearTimeout(timeoutId);
    if (error.name === 'AbortError') {
      throw new Error('Request timed out');
    }
    throw error;
  }
}

// ==================== TORRENT HANDLING ====================

async function handleFoundTorrents(data, tabId) {
  if (!data?.magnets?.length) return;

  // Update badge
  if (tabId) {
```

```
    await updateBadge(tabId, data.magnets.length);
  }

  // Cache in session storage
  await chrome.storage.session.set({
    [`torrents_${tabId}`]: data
  });

  // Auto-extract if enabled
  const config = await getConfig();
  if (config.autoExtract) {
    await handleExtraction(data);
  }
}

async function handleExtraction(data) {
  const keepAlive = setInterval(() => {
    chrome.runtime.getPlatformInfo(() => {});
  }, 20000);

  try {
    const result = await sendToApi('/api/torrents/extract', {
      source_url: data.url,
      title: data.title,
      magnets: data.magnets,
      torrent_files: data.torrentFiles || [],
      trackers: data.trackers || [],
      extracted_at: new Date().toISOString()
    });

    // Update stats
    const config = await getConfig();
    await updateConfig({
      totalExtracted: (config.totalExtracted || 0) +
        data.magnets.length,
      totalSent: (config.totalSent || 0) + 1,
      lastExtraction: new Date().toISOString()
    });

    // Show notification
    const cfg = await getConfig();
    if (cfg.showNotifications) {
      chrome.notifications.create({
        type: 'basic',
        iconUrl: 'icons/icon-48.png',
        title: 'Torrents Sent Successfully',
        message: `${data.magnets.length} magnet link(s) sent to
        API`
      });
    }

    return result;
```

```
  } catch (error) {
    chrome.notifications.create({
      type: 'basic',
      iconUrl: 'icons/icon-48.png',
      title: 'Failed to Send Torrents',
      message: error.message
    });
    throw error;
  } finally {
    clearInterval(keepAlive);
  }
}

// ==================== BADGE ====================

async function updateBadge(tabId, count) {
  if (count > 0) {
    await chrome.action.setBadgeText({
      text: count > 99 ? '99+' : String(count),
      tabId
    });
    await chrome.action.setBadgeBackgroundColor({ color:
        '#4CAF50', tabId });
  } else {
    await chrome.action.setBadgeText({ text: '', tabId });
  }
}

// ==================== STORAGE HELPERS ====================

const DEFAULT_CONFIG = {
  apiBaseUrl: 'http://localhost:8080',
  apiKey: '',
  timeout: 30000,
  autoExtract: false,
  showNotifications: true,
  notifyOnNewMagnets: true,
  badgeDisplay: 'count',
  totalExtracted: 0,
  totalSent: 0,
  lastExtraction: null
};

async function getConfig() {
  const { config } = await chrome.storage.local.get('config');
  return { ...DEFAULT_CONFIG, ...config };
}

async function updateConfig(updates) {
  const { config } = await chrome.storage.local.get('config');
  await chrome.storage.local.set({
    config: { ...config, ...updates }
```

```
  });
}

async function getStats() {
  const config = await getConfig();
  return {
    totalExtracted: config.totalExtracted,
    totalSent: config.totalSent,
    lastExtraction: config.lastExtraction
  };
}

async function cleanupOldSessionData() {
  // Session storage auto-clears on browser close,
  // but we can clean old entries here if needed
}
```

13.2 Popup (popup.js)

```
// popup.js
import { MSG_TYPES } from './shared/messages.js';

document.addEventListener('DOMContentLoaded', async () => {
  const [tab] = await chrome.tabs.query({ active: true,
        currentWindow: true });

  // Scan current page
  const scanBtn = document.getElementById('scan-btn');
  const sendBtn = document.getElementById('send-btn');
  const resultsDiv = document.getElementById('results');
  const statusDiv = document.getElementById('status');

  // Load cached data
  const cached = await chrome.storage.session.get(`torrents_$
        {tab.id}`);
  if (cached[`torrents_${tab.id}`]) {
    displayResults(cached[`torrents_${tab.id}`]);
  }

  scanBtn.addEventListener('click', async () => {
    try {
      scanBtn.disabled = true;
      scanBtn.textContent = 'Scanning...';

      const results = await chrome.tabs.sendMessage(tab.id, {
        action: 'SCAN_TORRENTS'
      });

      displayResults(results);
      statusDiv.textContent = `Found ${results.magnets.length}
        magnet(s)`;
    } catch (error) {
```

```
    statusDiv.textContent = 'Error: ' + error.message;
  } finally {
    scanBtn.disabled = false;
    scanBtn.textContent = 'Scan Page';
  }
});

sendBtn.addEventListener('click', async () => {
  try {
    sendBtn.disabled = true;
    sendBtn.textContent = 'Sending...';

    const results = await chrome.tabs.sendMessage(tab.id, {
      action: 'SCAN_TORRENTS'
    });

    const response = await chrome.runtime.sendMessage({
      type: MSG_TYPES.SEND_TORRENTS,
      data: results
    });

    if (response.success) {
      statusDiv.textContent = 'Sent successfully!';
    } else {
      statusDiv.textContent = 'Error: ' + response.error;
    }
  } catch (error) {
    statusDiv.textContent = 'Error: ' + error.message;
  } finally {
    sendBtn.disabled = false;
    sendBtn.textContent = 'Send to API';
  }
});

function displayResults(results) {
  resultsDiv.innerHTML = '';

  if (results.magnets.length === 0) {
    resultsDiv.innerHTML = '<p class="no-results">No torrent
      links found on this page.</p>';
    sendBtn.disabled = true;
    return;
  }

  sendBtn.disabled = false;

  const list = document.createElement('ul');
  list.className = 'torrent-list';

  results.magnets.slice(0, 10).forEach(magnet => {
    const li = document.createElement('li');
```

```
      const name = magnet.text || magnet.link.substring(0, 50) +
        '...';
      li.textContent = name;
      li.title = magnet.link;
      list.appendChild(li);
    });

    if (results.magnets.length > 10) {
      const more = document.createElement('li');
      more.textContent = `... and ${results.magnets.length - 10}
        more`;
      more.className = 'more';
      list.appendChild(more);
    }

    resultsDiv.appendChild(list);
  }
});
```

13.3 Options Page (options.js)

```
// options.js

document.addEventListener('DOMContentLoaded', async () => {
  // Load current config
  const config = await chrome.runtime.sendMessage({ type:
        'GET_CONFIG' });

  document.getElementById('api-url').value = config.apiBaseUrl ||
        '';
  document.getElementById('api-key').value = config.apiKey || '';
  document.getElementById('timeout').value = config.timeout ||
        30000;
  document.getElementById('auto-extract').checked =
        config.autoExtract || false;
  document.getElementById('show-notifications').checked =
        config.showNotifications !== false;

  // Save handler
  document.getElementById('save-btn').addEventListener('click',
        async () => {
    const updates = {
      apiBaseUrl: document.getElementById('api-url').value,
      apiKey: document.getElementById('api-key').value,
      timeout: parseInt(document.getElementById('timeout').value),
      autoExtract: document.getElementById('auto-
        extract').checked,
      showNotifications: document.getElementById('show-
        notifications').checked
    };

    await chrome.runtime.sendMessage({
      type: 'SET_CONFIG',
```

```
      data: updates
    });

    showStatus('Settings saved!', 'success');
  });

  // Test connection handler
  document.getElementById('test-btn').addEventListener('click',
        async () => {
    const url = document.getElementById('api-url').value;
    try {
      const response = await fetch(`${url}/health`, {
        method: 'GET',
        signal: AbortSignal.timeout(5000)
      });
      if (response.ok) {
        showStatus('Connection successful!', 'success');
      } else {
        showStatus(`Server error: ${response.status}`, 'error');
      }
    } catch (error) {
      showStatus(`Connection failed: ${error.message}`, 'error');
    }
  });

  function showStatus(message, type) {
    const status = document.getElementById('status');
    status.textContent = message;
    status.className = `status ${type}`;
    setTimeout(() => { status.textContent = ''; }, 3000);
  }
});
```

---

14. Permission Model Justification

Permission	Justification	User Warning
`storage`	Store API config, user preferences, statistics	None
`notifications`	Show extraction/ send status to user	None (OS may prompt separately)
`contextMenus`	Add right-click menu items for torrent extraction	None
`scripting`	Programmatic script injection for dynamic content	None

Permission	Justification	User Warning
`alarms`	Keep service worker alive, periodic tasks	None
`activeTab`	Temporary access to current tab when user clicks action	None
`host_permissions: http://*/ https://*/`	Scan web pages for torrent links; make API calls	“Read and change all your data on websites you visit”
`offscreen` (optional)	DOM parsing for complex torrent metadata extraction	None

Recommended Permission Strategy

Option A: Minimal (Recommended for Store) - Use `activeTab` + optional `host_permissions` requested at runtime - Users grant per-site access via click - No scary “read all data” warning at install

Option B: Full Access - Include `host_permissions: ["http://*/", "https://*/"]` in manifest - One-click operation on any page - Shows broad permission warning at install

Option C: Hybrid (Best UX) - Include common torrent site host_permissions (e.g., `*://1337x.to/*`, `*://thepiratebay.org/*`) - Use `activeTab` for general browsing - Offer runtime permission request for additional sites

---

15. Cross-Browser Compatibility Notes

Chrome (Chromium) vs Firefox Differences

Feature	Chrome/Edge/Opera	Firefox
Background	Service Worker	Event Page (non-persistent background page)
Manifest V3	Required (MV2 deprecated)	Supports both MV2 and MV3 [75]
`host_permissions`	Separate key	Also supported in MV3

Feature	Chrome/Edge/Opera	Firefox
`storage.session`	Available	Available
Offscreen Documents	Supported	Not yet supported
Blocking webRequest	Not in MV3	Still supported in MV3 [67]

Claim: Firefox supports both MV2 and MV3 with no immediate deprecation plans. Firefox uses non-persistent event pages instead of service workers in MV3, and continues to support blocking webRequest. **Source:** Mozilla Add-ons Community Blog **URL:** https://blog.mozilla.org/addons/2024/03/13/manifest-v3-manifest-v2-march-2024-update/ **Date:** 2024-03-13 **Excerpt:** "Firefox, however, has no plans to deprecate MV2 and will continue to support MV2 extensions for the foreseeable future… We continue to support DOM-based background scripts in the form of Event pages, and the blocking webRequest feature." **Confidence:** high

Cross-Browser Manifest

```
{
  "manifest_version": 3,
  "name": "Torrent Link Extractor",
  "version": "1.0.0",
  "browser_specific_settings": {
    "gecko": {
      "id": "torrent-extractor@example.com",
      "strict_min_version": "109.0"
    }
  },
  "background": {
    "service_worker": "service-worker.js"
  }
}
```

Firefox will use the `service_worker` field (supported since Firefox 121+) and falls back gracefully.

---

16. Magnet Link Detection Patterns

Regex Patterns

```
// Standard magnet link pattern
const MAGNET_REGEX = /magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32,40}/
        gi;
```

```javascript
// Full magnet URI with optional parameters
const MAGNET_FULL_REGEX = /magnet:\?xt=urn:btih:[a-f0-9]{40}
        (&[^\s"'<>]+)*/gi;

// .torrent file links
const TORRENT_FILE_REGEX = /\.torrent($|\?|&)/i;

// Common tracker announce URLs
const TRACKER_REGEX = /(udp|http|https):\/\/[^\s"'<>]+\/(announce|
        scrape)/gi;

// BitTorrent info hash (40-char hex)
const BTIH_REGEX = /[a-f0-9]{40}/gi;
```

Claim: Valid magnet links start with `magnet:?xt=urn:btih:` followed by a 40-character hexadecimal info hash. **Source:** Stack Overflow **URL:** https://stackoverflow.com/questions/8227280/any-way-to-verifiy-a-magnet-link-javascript **Date:** 2011-11-22 **Excerpt:** “The final regex, which actually works, is as follows: `/magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32}/i`” **Confidence:** high

---

17. Summary Checklist

Development Checklist

- [ ] manifest.json with `manifest_version: 3`
- [ ] Service worker registered at top-level
- [ ] All event listeners at top-level (not inside async)
- [ ] Content script with `run_at: "document_idle"`
- [ ] `host_permissions` declared for API and web access
- [ ] `storage` permission for configuration persistence
- [ ] `notifications` permission for status feedback
- [ ] `activeTab` for minimal permission model
- [ ] Messaging with `return true` for async responses
- [ ] Fetch with `AbortController` for timeout handling
- [ ]

- [ ] Service worker keep-alive via alarms API
- [ ] Badge updates on torrent detection
- [ ] Options page for server configuration
- [ ] Error handling and user notifications
- [ ] Context menu entries for quick actions

Key Gotchas

1. **Service workers die after 30s idle** - Use alarms API, not setInterval
2. **Listeners must be top-level** - Register synchronously, not in async callbacks
3. **return true in onMessage** - Required for async sendResponse
4. **No DOM in service worker** - Use offscreen documents for DOM parsing
5. **No Base64 in notifications** - Use file paths for icons
6. **CORS headers still apply** - Server must accept `chrome-extension://` origin
7. **User gesture for permissions** - `permissions.request()` must be in click handler
8. **Firefox differences** - Event pages vs service workers, MV2 still supported
9. **storage.session not persistent** - Use for transient SW state only
10. **Badge text max 4 chars** - Truncate counts > 999

References

1. Chrome Extension Documentation: https://developer.chrome.com/docs/extensions
2. MDN WebExtensions: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions
3. Service Worker Lifecycle: https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle
4. Message Passing: https://developer.chrome.com/docs/extensions/develop/concepts/messaging
5. Content Scripts: https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts
6. Offscreen Documents: https://developer.chrome.com/docs/extensions/reference/api/offscreen
7. Storage API: https://developer.chrome.com/docs/extensions/reference/api/storage

8. Action API: https://developer.chrome.com/docs/extensions/reference/api/action
9. Notifications API: https://developer.chrome.com/docs/extensions/reference/api/notifications
10. Permissions API: https://developer.chrome.com/docs/extensions/reference/api/permissions
11. Scripting API: https://developer.chrome.com/docs/extensions/reference/api/scripting
12. Alarms API: https://developer.chrome.com/docs/extensions/reference/api/alarms
13. Extension Packaging: https://developer.chrome.com/docs/extensions/how-to/distribute/install-extensions
14. Firefox MV3 Migration: https://extensionworkshop.com/documentation/develop/manifest-v3-migration-guide/
15. Manifest V3 Format: https://developer.chrome.com/docs/extensions/reference/manifest
16. Web Accessible Resources: https://developer.chrome.com/docs/extensions/reference/manifest/web-accessible-resources
17. CRX Update Format: https://developer.chrome.com/docs/apps/autoupdate
18. CORS in Extensions: https://developer.chrome.com/docs/extensions/develop/concepts/network-requests
19. Mozilla MV3 Update (Mar 2024): https://blog.mozilla.org/addons/2024/03/13/manifest-v3-manifest-v2-march-2024-update/
20. Mozilla reaffirms MV2 support: https://www.osnews.com/story/141805/mozilla-reaffirms-it-wont-remove-manifest-v2-support-from-firefox/